

DJANGO PLAYER PROJECT

Complete Revision Guide - Simple Step by Step

Model + CRUD + Search + Login/Signup/Logout + Template Inheritance + Custom 404

Main idea: Model stores data, View contains logic, URL connects the browser to a view, and Template shows the result.

1. Project Flow to Remember

```
Browser URL -> urls.py -> views.py -> models.py/database  
-> template.html -> browser output
```

- models.py: defines fields of the Player table.
- views.py: performs display, add, edit, delete, search, login and logout logic.
- urls.py: gives a URL path and name to every view.
- templates: contains HTML pages.
- settings.py: registers the app and controls DEBUG / ALLOWED_HOSTS.

2. Create Project and App

1. Open terminal in the folder where you want the project.
2. Create the Django project.
3. Enter the project folder.
4. Create the app.
5. Register the app in settings.py.

```
django-admin startproject playerproject7  
cd playerproject7  
python manage.py startapp playerapp
```

Register the app in settings.py

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
    'playerapp',  
]
```

Remember: Project folder contains settings.py and main urls.py. App folder contains models.py, views.py and admin.py.

3. Create Player Model

```
# playerapp/models.py
from django.db import models

class Player(models.Model):
    name = models.CharField(max_length=100)
    run = models.IntegerField()
    test = models.IntegerField()
    email = models.EmailField(unique=True)
    date = models.DateField()

    def __str__(self):
        return self.name
```

- CharField: stores short text.
- IntegerField: stores whole numbers.
- EmailField: checks email format.
- unique=True: same email cannot be saved twice.
- DateField: stores a date.
- __str__(): shows the player name in the admin panel.

Important: Use the same field names everywhere. If the model field is test, use p.test and form name="test". Do not mix test and test_innings.

4. Make and Apply Migrations

```
python manage.py makemigrations
python manage.py migrate
```

- makemigrations creates the migration instructions.
- migrate applies those instructions and creates/updates database tables.

5. Register Model in Admin

```
# playerapp/admin.py
from django.contrib import admin
from .models import Player

admin.site.register(Player)
```

Create admin user and run server

```
python manage.py createsuperuser
python manage.py runserver
```

Admin URL: Open `http://127.0.0.1:8000/admin/` and log in with the superuser account.

6. Create Templates Folder

```
playerapp/  
  templates/  
    nav.html  
    home.html  
    add.html  
    edit.html  
    details.html  
    welcome.html  
    signup.html  
    login.html  
    logout.html  
    404.html
```

7. Base Template and Template Inheritance

nav.html - common layout

```
<!DOCTYPE html>  
<html>  
<head>  
  <title>Player Project</title>  
</head>  
<body>  
  <a href="{% url 'home' %}">Home</a>  
  <a href="{% url 'welcome' %}">Welcome</a>  
  
  {% block content %}  
  {% endblock %}  
  
  <hr>  
  <footer>Copyright Chaitanya</footer>  
</body>  
</html>
```

Child page pattern

```
{% extends 'nav.html' %}  
  
{% block content %}  
  <!-- Only this page's different content -->  
{% endblock %}
```

Easy meaning: nav.html is the main/common page. The child page extends it and replaces only the block content area. Do not repeat html, head, body, navbar or footer in the child page.

8. Complete views.py

```
from django.shortcuts import render, redirect, get_object_or_404
from django.contrib.auth.forms import UserCreationForm, AuthenticationForm
from django.contrib.auth import login, logout
from .models import Player

def home(request):
    q = request.GET.get('q')
    if q:
        play = Player.objects.filter(name__icontains=q)
    else:
        play = Player.objects.all()
    return render(request, 'home.html', {'play': play})

def welcome(request):
    return render(request, 'welcome.html')

def add(request):
    if request.method == 'GET':
        return render(request, 'add.html')
    else:
        p = Player()
        p.name = request.POST.get('name')
        p.test = request.POST.get('test')
        p.run = request.POST.get('run')
        p.email = request.POST.get('email')
        p.date = request.POST.get('date')
        p.save()
        return redirect('home')

def edit(request, id):
    p = get_object_or_404(Player, pk=id)
    if request.method == 'GET':
        return render(request, 'edit.html', {'p': p})
    else:
        p.name = request.POST.get('name')
        p.test = request.POST.get('test')
        p.run = request.POST.get('run')
        p.email = request.POST.get('email')
        p.date = request.POST.get('date')
        p.save()
```

```

    return redirect('home')

def delete(request, id):
    p = get_object_or_404(Player, pk=id)
    p.delete()
    return redirect('home')

def details(request, id):
    p = get_object_or_404(Player, pk=id)
    return render(request, 'details.html', {'p': p})

def signup(request):
    if request.method == 'POST':
        form = UserCreationForm(request.POST)
        if form.is_valid():
            form.save()
            return redirect('login_view')
    else:
        form = UserCreationForm()
    return render(request, 'signup.html', {'form': form})

def login_view(request):
    if request.method == 'POST':
        form = AuthenticationForm(request, data=request.POST)
        if form.is_valid():
            user = form.get_user()
            login(request, user)
            return redirect('home')
    else:
        form = AuthenticationForm()
    return render(request, 'login.html', {'form': form})

def logout_view(request):
    logout(request)
    return render(request, 'logout.html')

def custom_404(request, exception):
    return render(request, '404.html', status=404)

```

9. Understand Every View Simply

home(): display and search

- request.GET.get('q') reads the text from the search input named q.

- if q means: if the user typed something.
- name__icontains=q finds names containing q and ignores uppercase/lowercase.
- objects.all() returns all players when search is empty.
- {'play': play} sends the result to home.html using the name play.

add(): create a new record

- GET opens the empty form.
- POST receives submitted values.
- Player() creates an empty Player object.
- request.POST.get(...) reads each form field.
- p.save() stores it in the database.
- redirect('home') returns to the home page.

edit(): update an existing record

- The URL sends an id.
- get_object_or_404 gets that player; invalid id shows 404.
- GET shows current values in edit.html.
- POST replaces values and p.save() updates the same row.

delete() and details()

- delete(): gets one player, deletes it, then redirects home.
- details(): gets one player and sends it to details.html as p.

signup, login and logout

- UserCreationForm creates a Django user.
- AuthenticationForm checks username and password.
- form.get_user() returns the verified user.
- login(request, user) starts the login session.
- logout(request) ends the session.

10. Main urls.py

```
# playerproject7/urls.py
from django.contrib import admin
from django.urls import path
from playerapp import views

urlpatterns = [
    path('admin/', admin.site.urls),
    path("", views.home, name='home'),
    path('add/', views.add, name='add'),
    path('welcome/', views.welcome, name='welcome'),
    path('edit/<int:id>/', views.edit, name='edit'),
    path('delete/<int:id>/', views.delete, name='delete'),
    path('details/<int:id>/', views.details, name='details'),
    path('signup/', views.signup, name='signup'),
    path('login/', views.login_view, name='login_view'),
    path('logout/', views.logout_view, name='logout_view'),
]

handler404 = 'playerapp.views.custom_404'
```

Remember: The name inside {% url ... %} must exactly match name= in urls.py. Example: name="login_view" means {% url "login_view" %}.

11. settings.py for Custom 404

```
DEBUG = False
ALLOWED_HOSTS = ['127.0.0.1', 'localhost']
```

- A custom 404 page normally appears only when DEBUG=False.
- When DEBUG=False, Django requires ALLOWED_HOSTS.
- 127.0.0.1 and localhost mean your own computer.
- Do not include :8000 inside ALLOWED_HOSTS.

12. HTML Templates

home.html

```
{% extends 'nav.html' %}

{% block content %}
<h1>Total Players: {{ play.count }}</h1>

<form method="GET">
  <input type="search" name="q">
  <button type="submit">Search</button>
</form>

<table border="1">
  <tr>
    <th>Name</th>
    <th>Test Innings</th>
    <th>Runs</th>
    <th>Email</th>
    <th>Date</th>
    <th>Actions</th>
  </tr>

  {% for i in play %}
  <tr>
    <td><a href="{% url 'details' i.id %}">{{ i.name }}</a></td>
    <td>{{ i.test }}</td>
    <td>
      {% if i.run > 500 %}
        <span style="color:green">{{ i.run }}</span>
      {% else %}
        <span style="color:red">{{ i.run }}</span>
      {% endif %}
    </td>
  </tr>
  {% endfor %}
</table>
```

```

        </td>
        <td>{{ i.email }}</td>
        <td>{{ i.date }}</td>
        <td>
            <a href="{% url 'edit' i.id %}">Edit</a>
            <a href="{% url 'delete' i.id %}">Delete</a>
        </td>
    </tr>
    {% empty %}
    <tr><td colspan="6">No player found</td></tr>
    {% endfor %}
</table>

<a href="{% url 'add' %}">Add Player</a>
<a href="{% url 'logout_view' %}">Logout</a>
{% endblock %}

```

add.html

```

{% extends 'nav.html' %}
{% block content %}
<h1>Add Player</h1>
<form method="POST">
    {% csrf_token %}
    <input type="text" name="name" placeholder="Player Name"><br>
    <input type="number" name="test" placeholder="Test Innings"><br>
    <input type="number" name="run" placeholder="Runs"><br>
    <input type="email" name="email" placeholder="Email"><br>
    <input type="date" name="date"><br>
    <button type="submit">Add Player</button>
</form>
{% endblock %}

```

edit.html

```

{% extends 'nav.html' %}
{% block content %}
<h1>Edit Player</h1>
<form method="POST">
    {% csrf_token %}
    <input type="text" name="name" value="{{ p.name }}"><br>
    <input type="number" name="test" value="{{ p.test }}"><br>
    <input type="number" name="run" value="{{ p.run }}"><br>
    <input type="email" name="email" value="{{ p.email }}"><br>
    <input type="date" name="date" value="{{ p.date|date:'Y-m-d' }}"><br>
    <button type="submit">Update Player</button>
</form>
{% endblock %}

```

details.html

```
{% extends 'nav.html' %}
{% block content %}
<h1>{{ p.name }}</h1>
<p>Test innings: {{ p.test }}</p>
<p>Runs: {{ p.run }}</p>
<p>Email: {{ p.email }}</p>
<p>Date: {{ p.date }}</p>
{% endblock %}
```

signup.html and login.html

```
<!-- signup.html -->
<form method="POST">
  {% csrf_token %}
  {{ form.as_p }}
  <button type="submit">Signup</button>
</form>

<!-- login.html -->
<form method="POST">
  {% csrf_token %}
  {{ form.as_p }}
  <button type="submit">Login</button>
</form>
```

logout.html

```
<h1>You are logged out</h1>
<a href="{% url 'login_view' %}">Login Again</a>
```

404.html

```
<h1>Page Not Found - Custom 404</h1>
<p>The requested page does not exist.</p>
<a href="{% url 'home' %}">Go Home</a>
```

13. Important Corrections from Your Screenshots

- In add.html, the title and button should say Add Player, not Edit Player.
- Use <input> for one-line values. Textarea also works, but input is simpler.
- Input is a void tag: write <input ...> and do not write </input>.
- Close span with , not .
- Use the same model field names in views and HTML.
- AuthenticationForm(request, data=request.POST) is correct.
- handler404 must be outside urlpatterns.
- custom_404 requires the exception parameter.

- The project URL name should not contain accidental spaces. Prefer login_view and logout_view.

14. Run and Test from Start to End

6. Save all files.
7. Run python manage.py makemigrations.
8. Run python manage.py migrate.
9. Run python manage.py runserver.
10. Open http://127.0.0.1:8000/.
11. Add a player and check that it appears in the table.
12. Click the player name to test details.
13. Click Edit and update the record.
14. Use the search box to search by part of the name.
15. Click Delete and verify the record disappears.
16. Open /signup/ and create an account.
17. Open /login/ and log in.
18. Open /logout/ and log out.
19. Keep DEBUG=False and open an invalid URL such as /abc/ to test custom 404.

15. Very Short Exam Memory Sheet

Task	Code to remember
Show all	Player.objects.all()
Search	Player.objects.filter(name__icontains=q)
Get one	get_object_or_404(Player, pk=id)
Save	p.save()
Delete	p.delete()
Read GET	request.GET.get('q')
Read POST	request.POST.get('name')
Open page	render(request, 'file.html', context)
Go to URL	redirect('home')
Dynamic URL	path('edit/<int:id>', views.edit, name='edit')
POST safety	{% csrf_token %}
Template loop	{% for i in play %} ... {% endfor %}
Template condition	{% if ... %} ... {% else %} ... {% endif %}
Base template	{% block content %} ... {% endblock %}
Child template	{% extends 'nav.html' %}

16. Final Checklist

- App added to INSTALLED_APPS.
- Model field names match views and HTML names.
- makemigrations and migrate completed.
- Model registered in admin.py.
- Every view has a matching URL.
- Every URL name matches the template {% url %} name.
- POST forms contain {% csrf_token %}.
- Base template contains block; child templates contain extends + block.
- DEBUG=False and ALLOWED_HOSTS set for custom 404.
- No unclosed HTML tags.

One-line summary: Create model -> migrate -> register admin -> write views -> connect URLs -> create templates -> run and test.